

21 | 移动测试神器：带你玩转Appium

2018-08-15 茹炳晟

软件测试52讲

[进入课程 >](#)



讲述：茹炳晟

时长 16:52 大小 7.73M



在上一篇文章中，我介绍了 Web App、Native App 和 Hybrid App 三种不同类型的移动应用以及对应的测试设计方法，也介绍了移动应用所特有的专项测试知识。

今天，我就以移动应用的自动化测试为主题，介绍目前主流的移动应用自动化测试框架 Appium。Appium 是一个开源的自动化测试框架，支持 iOS 和 Android 上 Web App、Native App 和 Hybrid App 的自动化测试。

由于基于 Appium 的移动应用环境搭建相对复杂，虽然网上也有不少教程，但是知识点都比较零碎，而且大多都是基于早期版本的示例，所以我会使用最新版本的 Appium Desktop 1.6.2 和 Appium Server 1.8.1 来展开今天的内容：

首先，我会展示如何在 Mac 环境下一步一步地搭建 Appium 测试环境；

接下来，我以 iOS 为例，实际开发两个测试用例，一个是 Native App 的测试用例，另一个是 Web App 的测试用例（因为 Hybrid App 的测试用例其实是类似的，Native App 的壳，Web App 的内容，所以就不再单独举例子了）；

然后，我会在 iOS 的模拟器上实际执行这两个测试用例（之所以选择 iOS 模拟器，而不用 iOS 真机做例子，是因为 iOS 真机的测试需要用到 Apple 开发者账号，还需要对被测应用进行签名等，会在环境搭建过程中引入很多额外步骤，而这些步骤对于讲解 Appium 并没有直接的关系）；

最后，当你已经通过实际使用对 Appium 形成感性认识后，我再来简单介绍一下 Appium 的内部原理，让你做到知其然知其所以然。

移动应用的自动化测试需求

在开始设计测试用例前，我们首先需要明确要开发的这两个自动化测试用例的具体测试需求。

1. Native App 的测试用例，被测 App 我选用了 Appium 官方的示例 App，被测 App 的源代码可以通过 [“https://github.com/appium/ios-test-app”](https://github.com/appium/ios-test-app) 下载，然后在 Xcode 中编译打包成 TestApp.app。
具体的测试需求是输入两个数字，然后点击 “Compute Sum” 验证两个数字相加后的结果是否正确。
2. Web App 的测试用例，具体需求是在 iPhone 上打开 Safari 浏览器，访问 Appium 的官方主页 [“http://appium.io”](http://appium.io)，然后验证主页的标题是否是 “Appium: Mobile App Automation Made Awesome”。

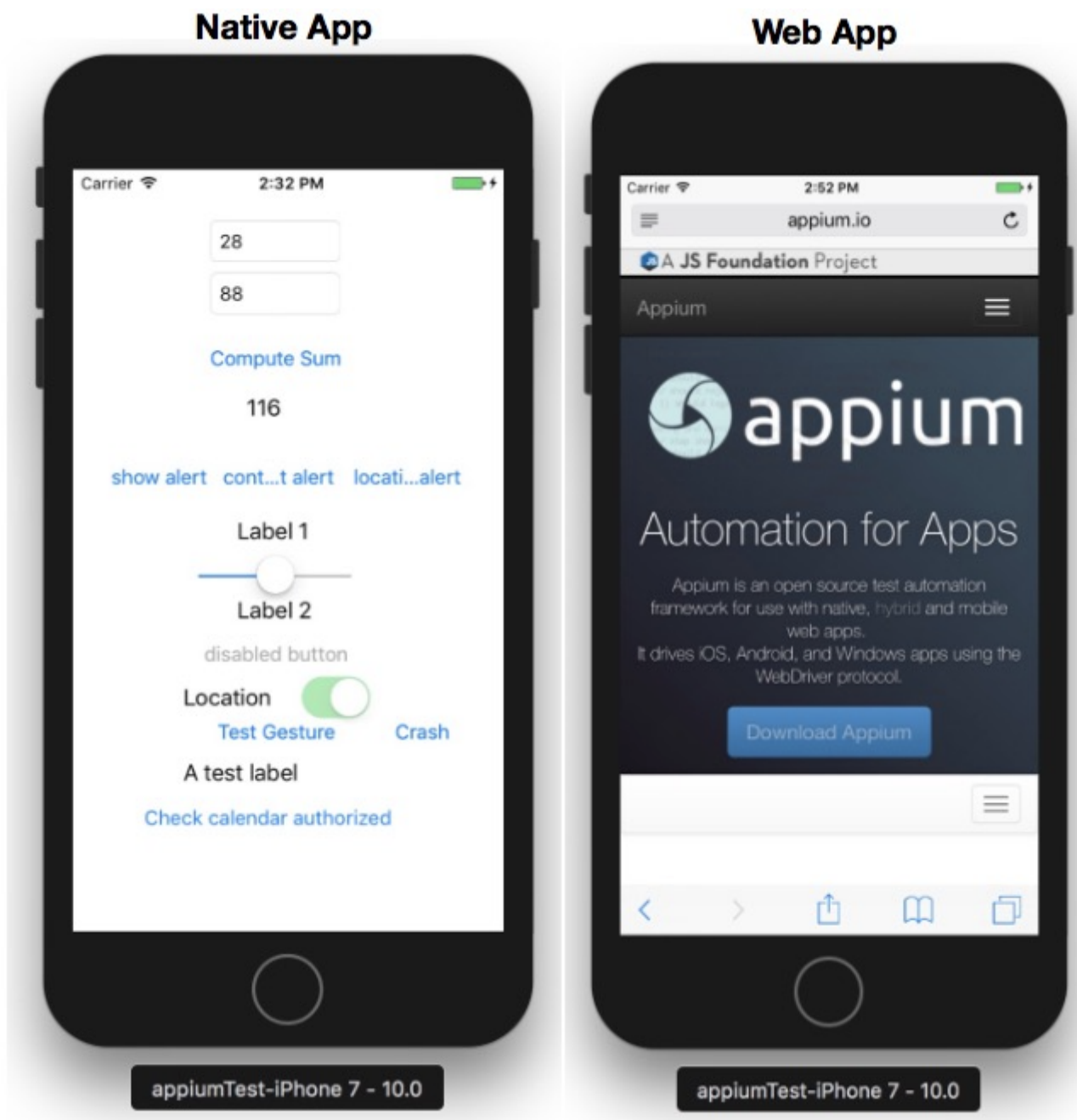


图 1 Native App 和 Web App 的 GUI 界面示例

接下来，我将从最初的环境搭建开始，和你来一起开发 iOS 上的 Native App 和 Web App 的测试用例。首先我们看一下 iOS 的环境搭建，如果你之前没有接触过这部分内容，你可以跟着我的步骤一步一步来做；而如果你已经比较熟悉 Xcode 的话，可以跳过这部分内容，直接从“Appium 环境搭建”部分看起。

iOS 环境搭建

在正式搭建 Appium 环境前，我们先来搭建 iOS 开发环境：

首先，下载安装 Xcode；

然后，在 Xcode 中下载 iOS 的模拟器；

接着，使用 Xcode 编译打包被测试 App；

最后，在 iOS 的模拟器中尝试手工执行这两个测试用例。

在 iOS 模拟器中，手动执行测试用例的具体操作步骤如下：

1. 启动 Xcode，导入 ios-test-app 下的 TestApp.xcodeproj 项目。
2. 在 Xcode 中，打开 “Preferences” 中的 “Components”，完成 iOS 10.0 Simulator 的下载。
3. 在 Xcode 的 “General” 页面，将 TestApp 的 “Deployment Target” 设置为 10.0，并且将 “Devices” 设置为 “iPhone”，如图 2 所示。

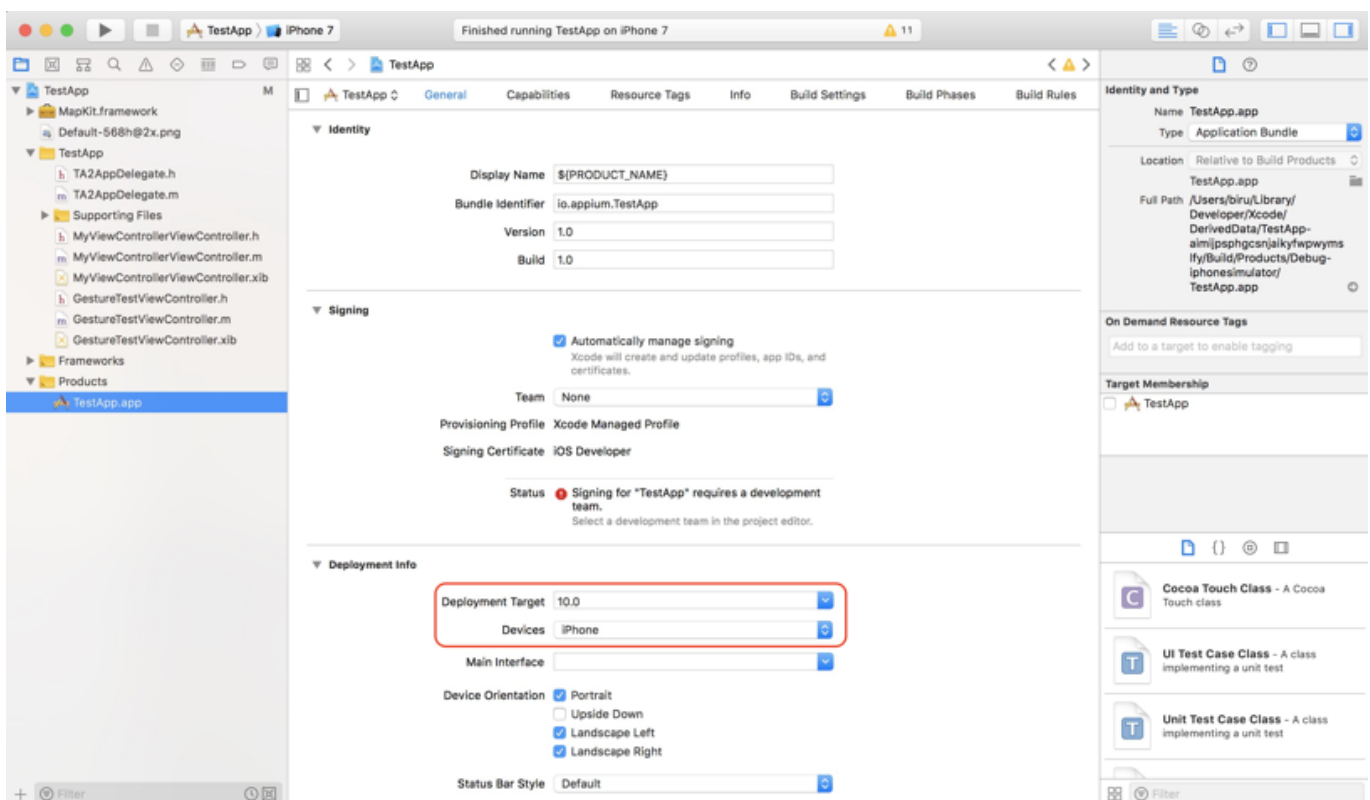


图 2 TestApp 的 General 配置

4. 在 Xcode 中编译运行 TestApp，之后系统会自动启动 iPhone 模拟器，自动完成 TestApp 的安装，并在 iPhone 模拟器中自动启动 TestApp。
5. 在 TestApp 中手动执行自定义的加法测试用例。
6. 退出 TestApp，然后打开 Safari 浏览器，在 Safari 中执行访问 Appium 官方主页的测试用例。

至此，你已经搭建好了 iOS 开发环境，并且成功编译打包了 TestApp。接下来，我们再一起来搭建 Appium 测试环境，并尝试在 Appium 中开发上述的两个测试用例。

Appium 测试环境搭建

通过 Appium 的官方网站下载并安装最新版本的 Appium，截止本文写作的时间，最新版本是 Appium-1.6.2.dmg。

需要注意的是，早期版本和网上很多教程都建议用命令行的形式启动 Appium Server，但在这里我是想强调的是，你完全可以通过界面启动（在 Launchpad 中找到 Appium 的图标，点击即可启动），而且新版本的 Appium 也推荐这个启动方式。通过界面启动，是目前最简单直接的方式。

然后，你需要用命令行 “npm install -g appium-doctor” 安装 Appium 的环境诊断工具 appium-doctor，用于检查 Appium 所依赖的相关环境变量以及其他安装包是否都已经配置好了。如果还没有，就需要逐个安装，并根据 appium-doctor 的提示配置环境变量。

这里，Appium 最主要的依赖项主要有：Java、Node.js、Xcode、Carthage、Android SDK、adb 等。如果你所有的环境依赖都正常配置的话，你就会看到 appium-doctor 返回这样一个截图，如图 3 所示。

```
LM-SHC-16501497:~ biru$ appium-doctor
info AppiumDoctor Appium Doctor v.1.4.3
info AppiumDoctor ### Diagnostic starting ###
info AppiumDoctor   ✓ The Node.js binary was found at: /usr/local/bin/node
info AppiumDoctor   ✓ Node version is 9.11.1
info AppiumDoctor   ✓ Xcode is installed at: /Applications/Xcode.app/Contents/Developer
info AppiumDoctor   ✓ Xcode Command Line Tools are installed.
info AppiumDoctor   ✓ DevToolsSecurity is enabled.
info AppiumDoctor   ✓ The Authorization DB is set up properly.
info AppiumDoctor   ✓ Carthage was found at: /usr/local/bin/carthage
info AppiumDoctor   ✓ HOME is set to: /Users/biru
info AppiumDoctor   ✓ ANDROID_HOME is set to: /Users/biru/Library/Android/sdk
info AppiumDoctor   ✓ JAVA_HOME is set to: /Library/Java/JavaVirtualMachines/jdk1.8.0_171.jdk/Contents/Home
info AppiumDoctor   ✓ adb exists at: /Users/biru/Library/Android/sdk/platform-tools/adb
info AppiumDoctor   ✓ android exists at: /Users/biru/Library/Android/sdk/tools/android
info AppiumDoctor   ✓ emulator exists at: /Users/biru/Library/Android/sdk/tools/emulator
info AppiumDoctor   ✓ Bin directory of $JAVA_HOME is set
info AppiumDoctor ### Diagnostic completed, no fix needed. ###
info AppiumDoctor
info AppiumDoctor Everything looks good, bye!
info AppiumDoctor
LM-SHC-16501497:~ biru$
```

图 3 正常配置环境依赖后，appium-doctor 返回的截图

按照上面的步骤，配置好 Appium 的环境依赖后，就可以继续启动 Appium Server 了。

Appium Inspector 的使用

为了后续测试用例的顺利执行，我们可以先来熟悉一下 Appium Inspector 的使用。Appium Inspector 主要是用来协助对界面元素进行定位的工具。

首先，我们来看看如何使用 Appium Inspector 启动 iPhone 的模拟器，并在模拟器上运行 TestApp，以及如何通过 Inspector 定位 TestApp 界面上的元素（了解元素的定位是后续开发自动化脚本的基础）。具体的操作过程如下。

1. 通过 Appium Server 的 “Start Inspector Session” 按钮，进入 Session 配置界面。

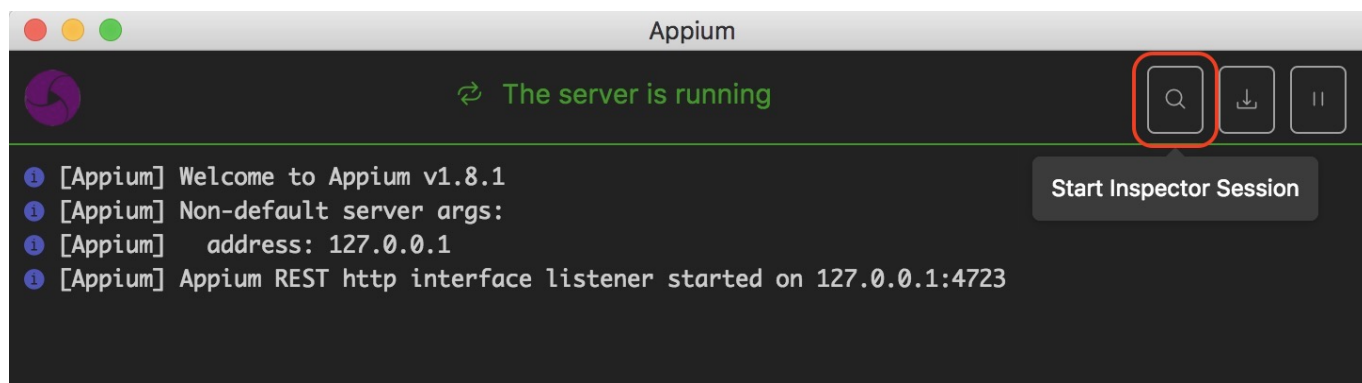


图 4 点击 “Start Inspector Session” 按钮打开 Session 配置界面

2. 在 Session 配置界面完成必要参数的配置。这里你需要根据选用的移动设备操作系统、模拟器 / 真机等具体情况来完成参数配置工作。需要配置参数主要包括：
platformName、platformVersion、DeviceName、automationName 和 app。
其中，automationName，指自动化测试框架的名称，这里采用了 XCUITest；app 指被测 Native App 的安装包路径，这里使用之前 Xcode 打包生成的 TestApp.app，这样启动模拟器时，就会自动把 TestApp.app 安装到模拟器中。
其他参数的配置非常简单，我就不再一一展开了。

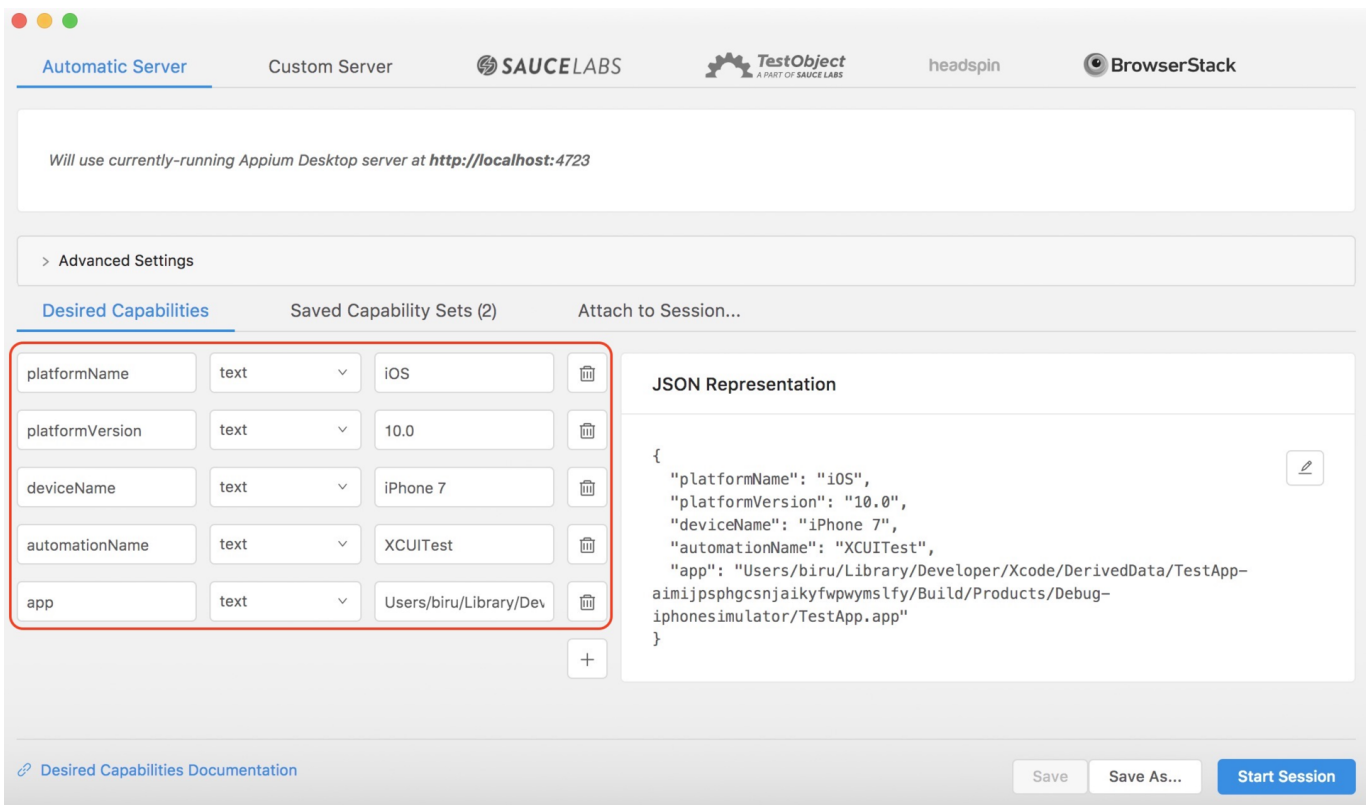


图 5 Session 配置界面

3. 完成配置后，点击 Session 界面的“Start Session”按钮，启动 iPhone 模拟器，并在 iPhone 模拟器中启动 TestApp，同时还会打开 Inspector 窗口。如图 6 所示。

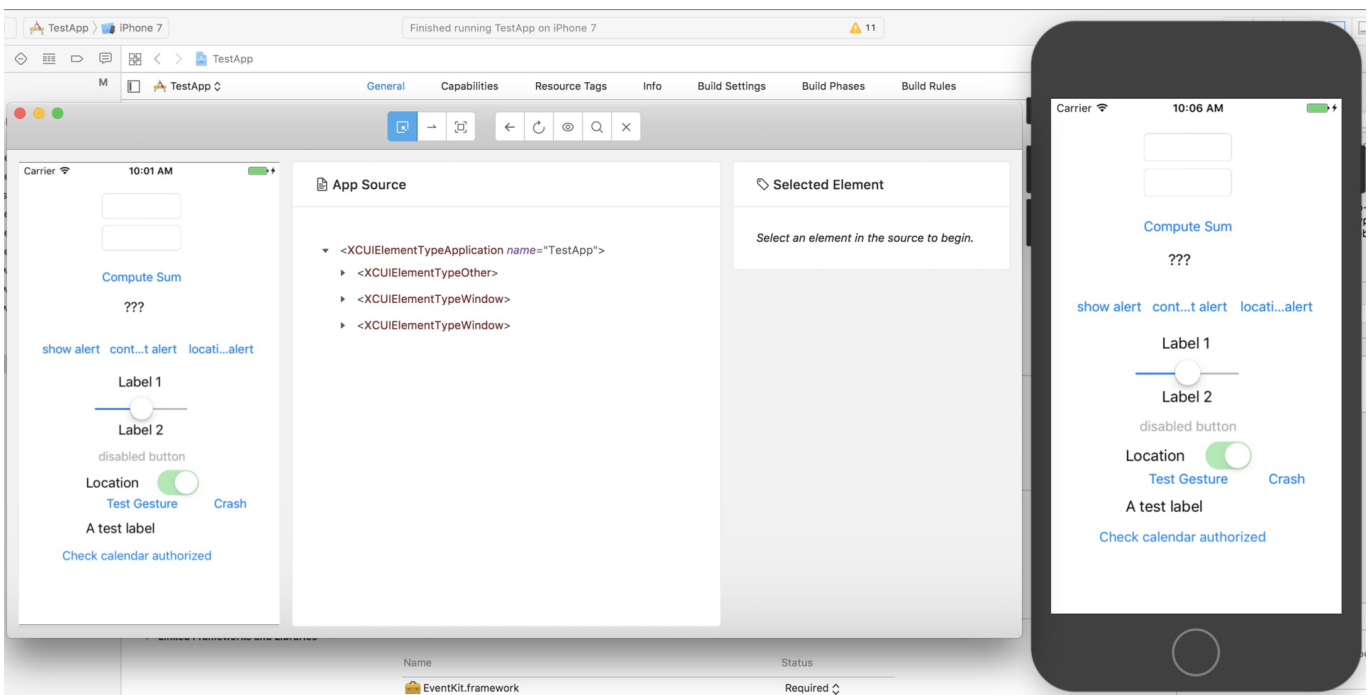


图 6 启动 Session 后的 Inspector 窗口

4. 在 Inspector 窗口，我们可以利用 “Select Elements” 功能，通过点击元素显示 Native App 上的元素定位信息。如图 7 所示。

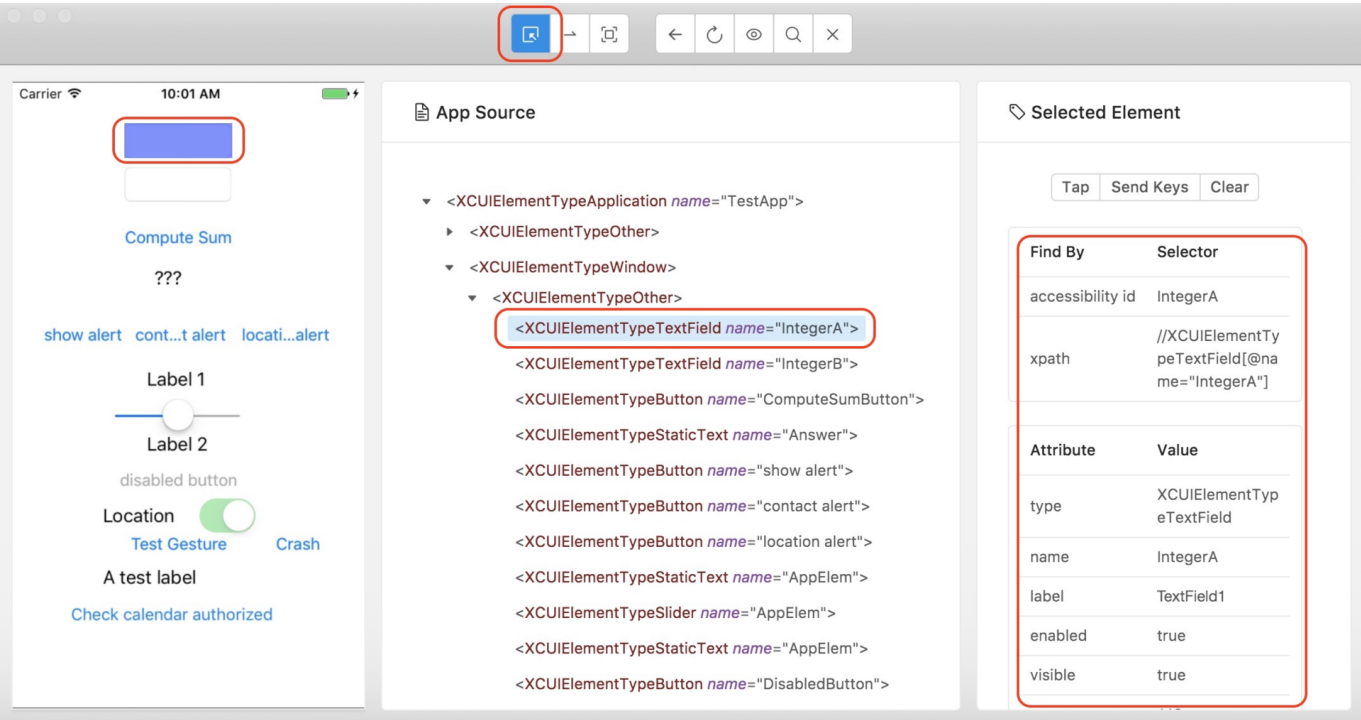


图 7 “Select Elements” 功能示例

5. 在 Inspector 窗口，可以通过 “Recording” 功能生成不同语言的自动化脚本。比如在启用了 “Recording” 功能后，点击 “Compute Sum” 按钮，就会生成如图 8 所示的自动化脚本片段。

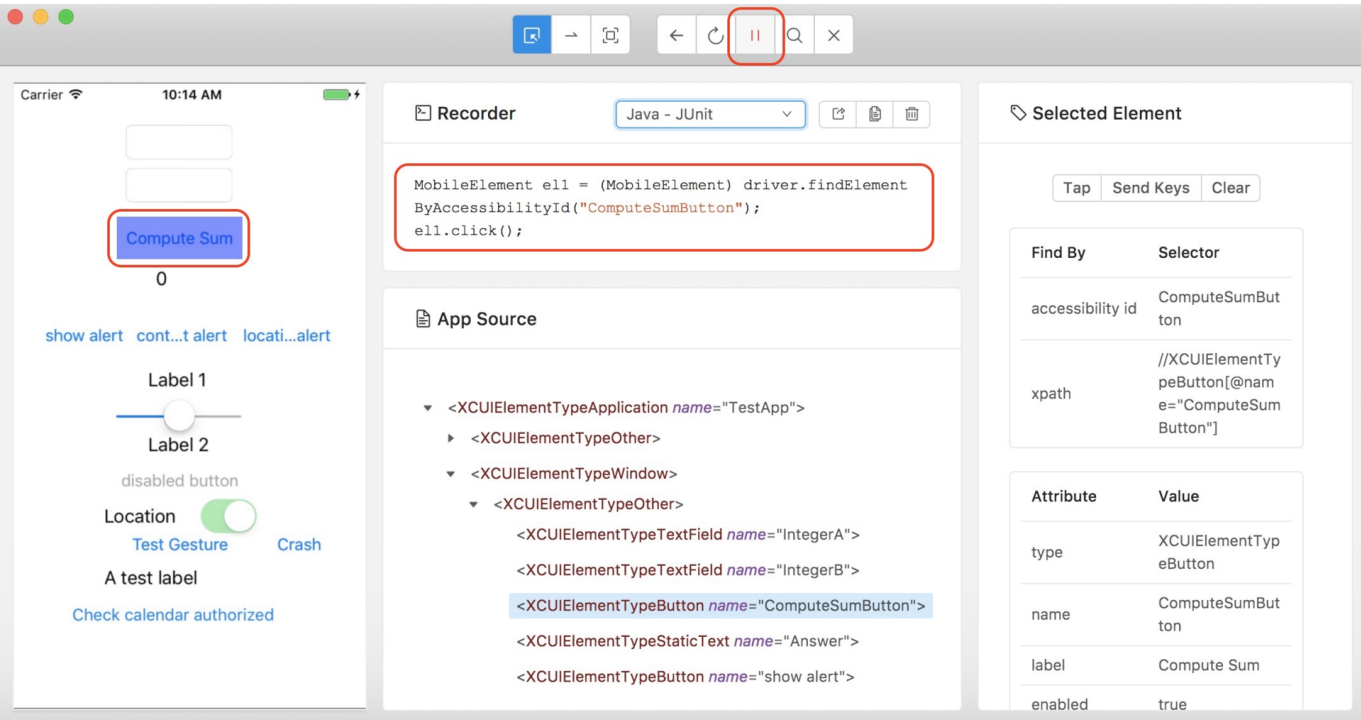


图 8 “Recording” 功能示例

了解了如何通过 Inspector 获取元素定位信息的方法之后，我们就来正式开发基于 Appium 的第一个 Web App 和第一个 Native App 的测试用例。

基于 Appium 开发你的第一个 Native App 的测试用例

第一步，建立一个空的 Maven 项目，然后在 POM 文件中加入如图 9 所示的依赖。

在这个案例里面，我们会使用 TestNG 组织测试用例，所以代码的第 14 行加入了 TestNG 的依赖。

第 19 行的 java-client 是关键，java-client 的作用是利用 Java 代码将测试用例中的操作步骤发送给 Appium Server，然后由 Appium Server 自动完成这些操作。

目前 Appium 支持多种编程语言，每种语言都有自己的 client，比如这里使用 Java 语言，所以引入了 java-client；如果你使用 Python 语言，那么就需要引用 python-client。

A screenshot of a Maven POM file in a code editor. The file is an XML document with the following structure:

```
1 <?xml version="1.0" encoding="UTF-8"?>
2 <project xmlns="http://maven.apache.org/POM/4.0.0"
3     xmlns:xsi="http://www.w3.org/2001/XMLSchema-instance"
4     xsi:schemaLocation="http://maven.apache.org/POM/4.0.0 http://maven.apache.org/xsd/maven-4.0.0.xsd">
5     <modelVersion>4.0.0</modelVersion>
6
7     <groupId>appium</groupId>
8     <artifactId>appiumdemo</artifactId>
9     <version>1.0-SNAPSHOT</version>
10
11     <dependencies>
12         <dependency>
13             <groupId>org.testng</groupId>
14             <artifactId>testng</artifactId>
15             <version>6.14.3</version>
16         </dependency>
17         <dependency>
18             <groupId>io.appium</groupId>
19             <artifactId>java-client</artifactId>
20             <version>6.1.0</version>
21         </dependency>
22     </dependencies>
23
24 </project>
```

The dependencies section, starting from line 11 and ending at line 22, is highlighted with a red rounded rectangle. This section contains two dependencies: TestNG (version 6.14.3) and the Appium Java client (version 6.1.0).

图 9 POM 文件加入 TestNG 和 java-client 的依赖

第二步，创建一个类，并命名为 “iOS_NativeApp_DemoTest”，然后按照如图 10 所示的代码实现这个 class。

注意，这里的代码是真实的可执行 Java 代码，你可以直接拿去使用。


```

1  import java.net.MalformedURLException;
2  import java.net.URL;
3
4  import io.appium.java_client.MobileElement;
5  import org.openqa.selenium.remote.DesiredCapabilities;
6  import org.testng.Assert;
7  import org.testng.annotations.AfterTest;
8  import org.testng.annotations.BeforeTest;
9  import org.testng.annotations.Test;
10
11  import io.appium.java_client.ios.IOSDriver;
12  import io.appium.java_client.remote.MobileCapabilityType;
13
14  /**
15   * iOS Native App Test Demo.
16   */
17  public class iOS_NativeApp_DemoTest
18  {
19      public static IOSDriver iosdriver;
20
21      @BeforeTest
22      public void beforeTest( ) throws MalformedURLException
23      {
24          DesiredCapabilities capabilities = new DesiredCapabilities();
25          capabilities.setCapability(MobileCapabilityType.APPIUM_VERSION, value: "1.8.1");
26          capabilities.setCapability(MobileCapabilityType.PLATFORM_VERSION, value: "10.0");
27          capabilities.setCapability(MobileCapabilityType.PLATFORM_NAME, value: "iOS");
28          capabilities.setCapability(MobileCapabilityType.AUTOMATION_NAME, value: "XCUITest");
29          capabilities.setCapability(MobileCapabilityType.DEVICE_NAME, value: "iPhone 7");
30          capabilities.setCapability(MobileCapabilityType.APP,
31              value: "/Users/biru/Library/Developer/Xcode/DerivedData/" +
32                  "TestApp-aimjpsphgcsnialkyfwpvmslfy/Build/Products/Debug-iphonesimulator/TestApp.app");
33          capabilities.setCapability("newCommandTimeout", 2000);
34
35          iosdriver = new IOSDriver(new URL( spec: "http://127.0.0.1:4723/wd/hub"), capabilities);
36      }
37
38      @AfterTest
39      public void afterTest( )
40      {
41          iosdriver.quit();
42      }
43
44      @Test
45      public static void launchNativeAppTest() throws InterruptedException{
46          //Web element definition
47          MobileElement integerA = (MobileElement) iosdriver.findElementByAccessibilityId( using: "IntegerA");
48          MobileElement integerB = (MobileElement) iosdriver.findElementByAccessibilityId( using: "IntegerB");
49          MobileElement computeSumButton = (MobileElement) iosdriver.findElementByAccessibilityId( using: "ComputeSumButton");
50          MobileElement answer = (MobileElement) iosdriver.findElementByAccessibilityId( using: "Answer");
51
52          //Test operation
53          integerA.sendKeys( ...keysToSend: "28");
54          integerB.sendKeys( ...keysToSend: "88");
55          computeSumButton.click();
56
57          //Test result assert
58          Assert.assertEquals(answer.getText().toString(), s1: "116", s2: "aculate sum error");
59      }
60  }

```

图 10 Native App 测试用例实例

代码第 21 行的 @BeforeTest，第 38 行的 @AfterTest，以及第 44 行的 @Test，都是利用了 TestNG 的 annotation 对函数进行标注。

标有 @Test 的函数是真正的测试主体，所有测试相关的步骤都放在这个函数中；

标有 @ BeforeTest 的函数会在 @Test 函数之前执行测试的相关准备工作，图中的代码用这个函数完成了 DesiredCapabilities 的设置，并用该 Capabilities 构造了 iosdriver；

标有 @ AfterTest 的函数在 @Test 函数执行结束后执行，主要用于环境的清理和收尾，图示的代码用这个函数完成了 iosdriver 的退出操作。

代码的第 24-33 行构造了 DesiredCapabilities 对象，并对 APPIUM_VERSION、PLATFORM_VERSION、PLATFORM_NAME、AUTOMATION_NAME、DEVICE_NAME 和 APP 等参数进行了设置。其中 APP 的值是被测 Native App 安装包的绝对路径。

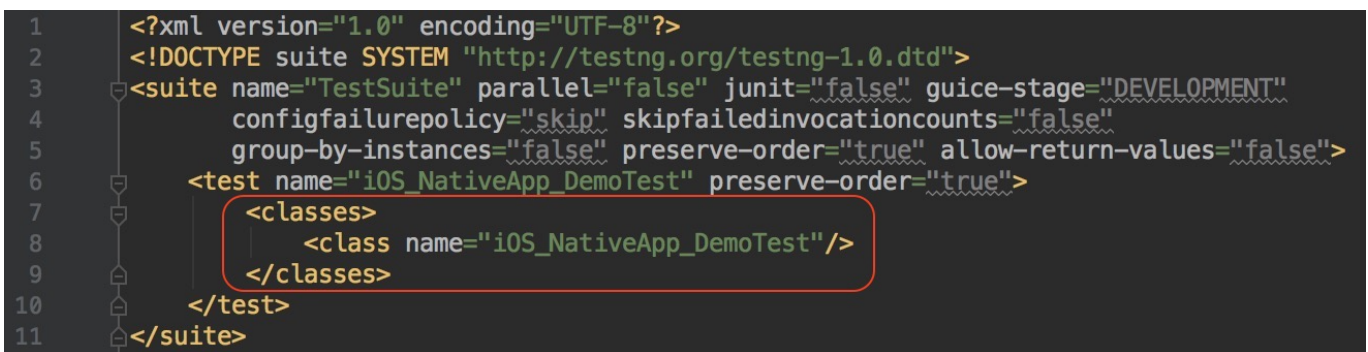
代码的第 46-58 行是测试用例的主体部分，主要分为三部分：

第 47-50 行通过 iosdriver 的 findElementByAccessibilityId 方法定义了页面上的四个元素，分别是输入参数框 A、输入参数框 B、计算按钮和加法结果显示框。代码中具体的 AccessibilityId 可以通过 Inspector 获取。

第 53-55 行通过自定义元素的操作执行加法运算。

第 58 行通过断言方法 assertEquals 验证加法运算的结果。

第三步，为了运行这个 TestNG 的测试用例，我们需要再添加一个 testng.xml 文件，具体内容如图 11 所示。



```
1  <?xml version="1.0" encoding="UTF-8"?>
2  <!DOCTYPE suite SYSTEM "http://testng.org/testng-1.0.dtd">
3  <suite name="TestSuite" parallel="false" junit="false" guice-stage="DEVELOPMENT"
4      configfailurepolicy="skip" skipfailedinvocationcounts="false"
5      group-by-instances="false" preserve-order="true" allow-return-values="false">
6      <test name="iOS_NativeApp_DemoTest" preserve-order="true">
7          <classes>
8              <class name="iOS_NativeApp_DemoTest"/>
9          </classes>
10     </test>
11 </suite>
```

图 11 testng.xml 文件示例

第四步，在保证 Appium Server 已经启动的情况下，就可以运行 testng.xml 执行测试了。测试开始后，首先会自动启动基于 iOS 10.0 的 iPhone 7 模拟器，然后依次自动完成 WebDriverAgent (WDA) 和被测 Native App 的安装。

WDA 是由 Facebook 开源的支持 iOS 自动化的代理工具，其底层通过 XCUItest 实现自动化。

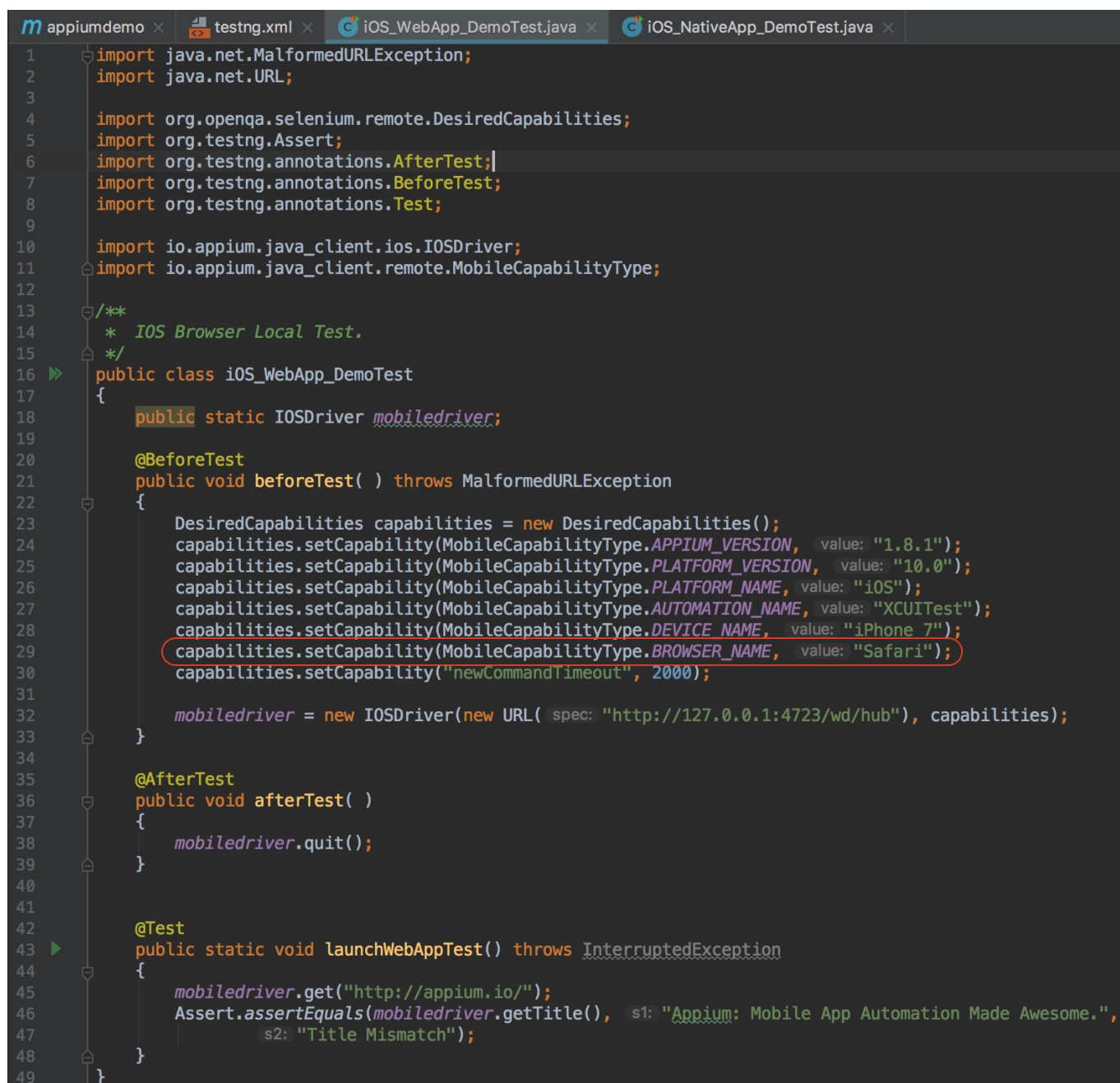
接着，就会自动运行被测 Native App，并根据 @Test 函数中定义的步骤完成自动化测试的步骤和验证。

到此，我们的第一个基于 Appium 的 Native App 自动化测试用例就设计完了。

基于 Appium 开发你的第一个 Web App 的测试用例

有了 Native App 测试用例的设计基础，再来实现一个基于 Appium 的 Web App 自动化测试用例就简单得多了。

第一步，在上述的 Maven 项目中再创建一个类，并命名为“iOS_WebApp_DemoTest”，然后按照如图 12 所示的代码实现这个类。



```
1 import java.net.MalformedURLException;
2 import java.net.URL;
3
4 import org.openqa.selenium.remote.DesiredCapabilities;
5 import org.testng.Assert;
6 import org.testng.annotations.AfterTest;
7 import org.testng.annotations.BeforeTest;
8 import org.testng.annotations.Test;
9
10 import io.appium.java_client.ios.IOSDriver;
11 import io.appium.java_client.remote.MobileCapabilityType;
12
13 /**
14  * IOS Browser Local Test.
15  */
16 public class iOS_WebApp_DemoTest
17 {
18     public static IOSDriver mobiledriver;
19
20     @BeforeTest
21     public void beforeTest() throws MalformedURLException
22     {
23         DesiredCapabilities capabilities = new DesiredCapabilities();
24         capabilities.setCapability(MobileCapabilityType.APPIUM_VERSION, "1.8.1");
25         capabilities.setCapability(MobileCapabilityType.PLATFORM_VERSION, "10.0");
26         capabilities.setCapability(MobileCapabilityType.PLATFORM_NAME, "iOS");
27         capabilities.setCapability(MobileCapabilityType.AUTOMATION_NAME, "XCUITest");
28         capabilities.setCapability(MobileCapabilityType.DEVICE_NAME, "iPhone 7");
29         capabilities.setCapability(MobileCapabilityType.BROWSER_NAME, "Safari");
30         capabilities.setCapability("newCommandTimeout", 2000);
31
32         mobiledriver = new IOSDriver(new URL("http://127.0.0.1:4723/wd/hub"), capabilities);
33     }
34
35     @AfterTest
36     public void afterTest()
37     {
38         mobiledriver.quit();
39     }
40
41     @Test
42     public static void launchWebAppTest() throws InterruptedException
43     {
44         mobiledriver.get("http://appium.io/");
45         Assert.assertEquals(mobiledriver.getTitle(), s1: "Appium: Mobile App Automation Made Awesome.",
46                             s2: "Title Mismatch");
47     }
48 }
49
```

图 12 Web App 测试用例实例

代码的整体结构和上述 Native App 测试用例的完全一致，只有一个地方需要特别注意：代码的第 29 行，由于 Web App 是基于浏览器的测试，所以这里不需要指定 App 这个参

数，而是直接用 BROWSER_NAME 指定浏览器的名字即可。

对于测试用例的主体部分，也就是代码的第 45-47 行就比较简单了，首先打开 Safari 浏览器并访问 “<http://appium.io/>”，接着用断言方法 assertEquals 验证页面的 Title 是不是 “Appium: Mobile App Automation Made Awesome.”。其中，实际页面的 Title，可以通过 mobiledriver 的 getTitle 方法获得。

第二步，在 testng.xml 中添加这个 Web App 的测试用例，然后我们就可以在 Appium Server 已经启动的情况下执行这个测试用例了。

这个测试用例，首先会自动启动基于 iOS 10.0 的 iPhone 7 模拟器，然后自动打开 Safari 浏览器并访问 Appium 的官方网站。执行完成后的界面如下图 13 所示。

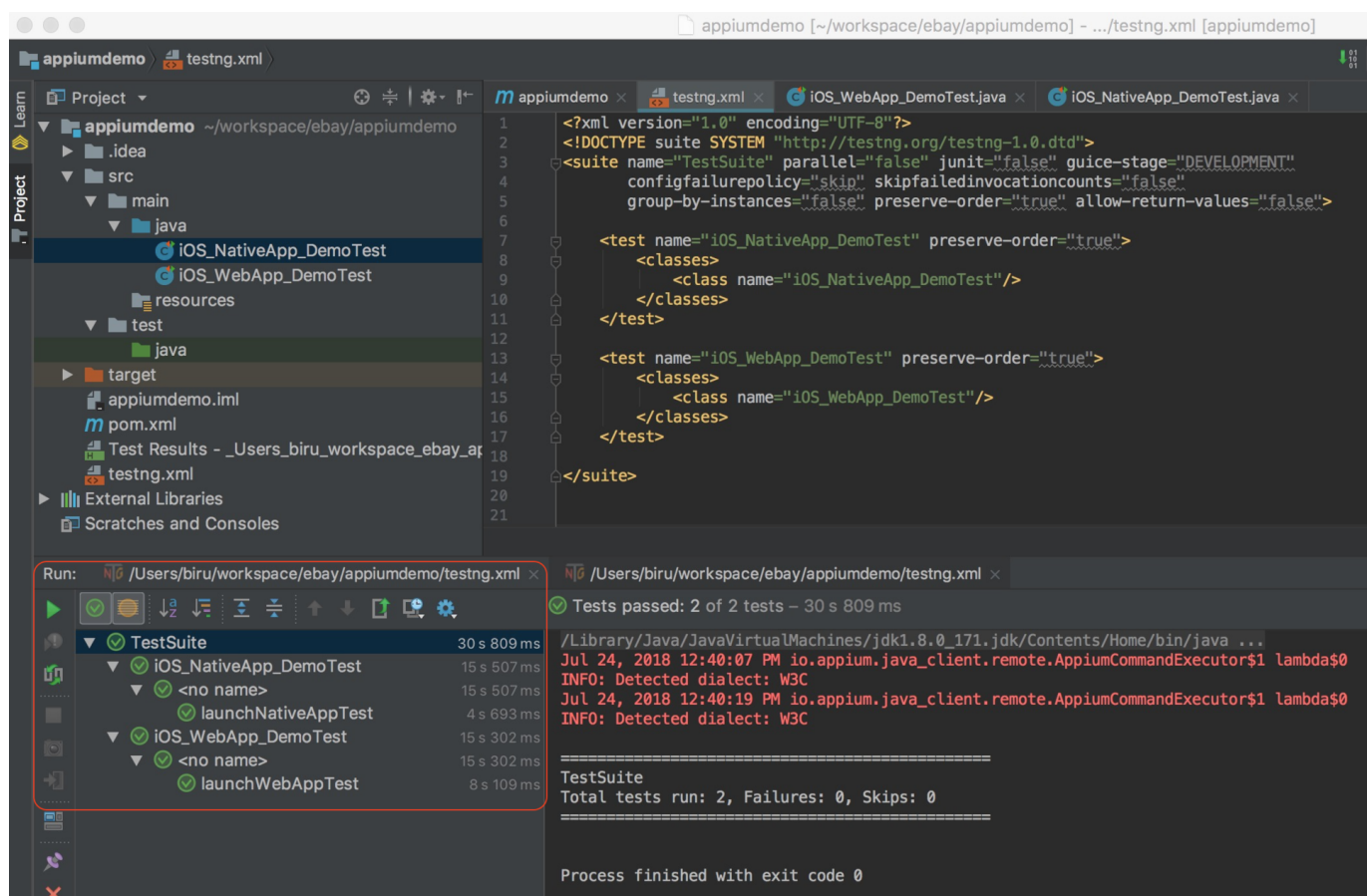


图 13 测试用例执行完成的界面

进行到这里，我们基于 Appium 开发的第一个 Web App 的自动化测试用例，也就开发完成了。

经过前面 Appium 环境搭建，以及两个测试用例的设计，相信你已经对 Appium 有了一个感性的认识了。那么，Appium 的实现原理又是怎样的呢？理解了 Appium 的使用原理，可以帮助你更好地使用这个工具，设计更加“有的放矢”的测试用例。

Appium 的实现原理

Appium 作为目前主流的移动应用自动化测试框架，具有极强的灵活性，主要体现在以下 5 个方面：

测试用例的实现支持多种编程语言，比如 Java、Ruby、Python 等；

Appium Server 支持多平台，既有基于 Mac 的版本，也有基于 Windows 的版本；

支持 Web App、Native App 和 Hybrid App 三大类移动应用的测试；

既支持 iOS，也支持 Android；

既支持真机，也支持模拟器。

实际应用中，你可以根据项目情况灵活组合完成移动应用的自动化测试。比如，用 Java 写 iOS 上的 Native App 的测试用例，测试用例跑在 Mac 平台的 iPhone 虚拟机上；或者，用 Python 写 Android 上的 Web App 的测试用例，测试用例通过 Windows 平台跑在 Android 的真机上。

这样的组合还有很多很多。那你有没有想过，Appium 为什么可以做到如此强大的灵活性呢？这就要从 Appium 的基本原理讲起了。

要真正理解 Appium 的内部原理，你可以把 Appium 分成三大部分，分别是 Appium Client、Appium Server 和设备端。这三部分的关系如图 14 所示。

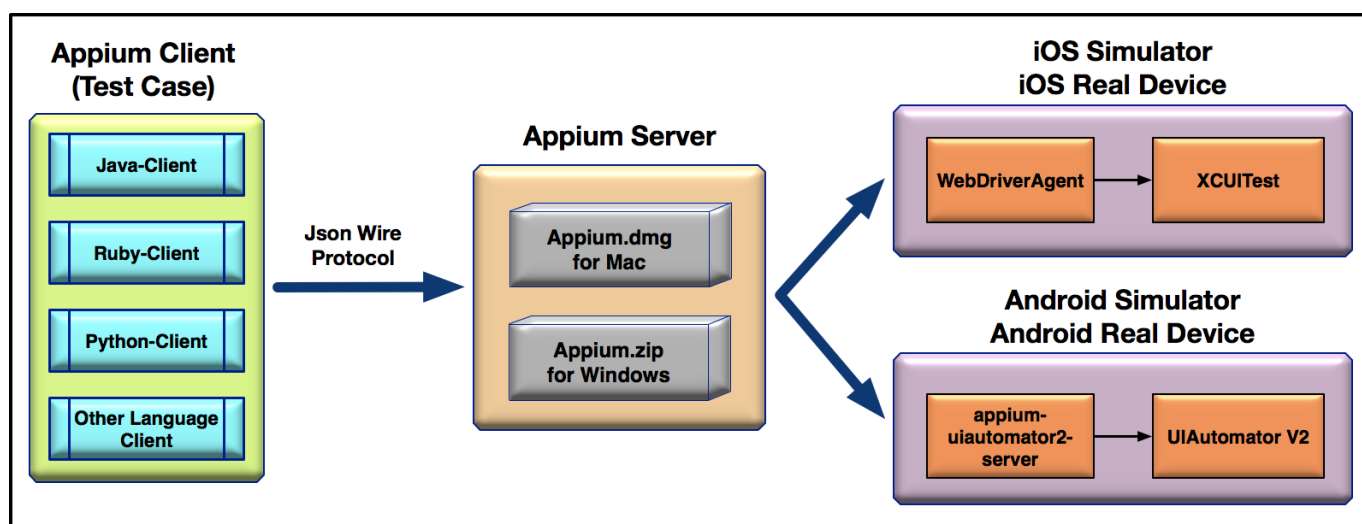


图 14 Appium 内部原理

我们先来看看处于中间位置的 Appium Server。

Appium Server 有 Mac 和 Windows 版本，也就是说 Appium Server 可以运行在 Mac 或者 Windows 电脑上。本质上，Appium Server 是一个 Node.js 应用，接受来自 Appium Client 的请求，解析后通过 WebDriver 协议和设备端上的代理打交道。

如果是 iOS，Appium Server 会把操作请求发送给 WebDriverAgent（简称 WDA），然后 WDA 再基于 XCUITest 完成 iOS 模拟器或者真机上的自动化操作；

如果是 Android，Appium Server 会把操作请求发送给 appium-UiAutomator2-server，然后 appium-UiAutomator2-server 再基于 UIAutomator V2 完成 Android 模拟器或者真机上的自动化操作。

Appium Client 其实就是测试代码，使用对应语言的 Client 将基于 JSON Wire 协议的操作指令发给 Appium Server。

整体来说，Appium 的内部原理可以总结为：**Appium 属于 C/S 架构，Appium Client 通过多语言支持的第三方库向 Appium Server 发起请求，基于 Node.js 的 Appium Server 会接受 Appium Client 发来的请求，接着和 iOS 或者 Android 平台上的代理工具打交道，代理工具在运行过程中不断接收请求，并根据 WebDriver 协议解析出要执行的操作，最后调用 iOS 或者 Android 平台上的原生测试框架完成测试。**

总结

好了，我来总结一下今天的主要内容：

目前网络上，Appium 工具使用相关的资料都比较零散，为此我以最新版本的 Appium Desktop 1.6.2 和 Appium Server 1.8.1 为例，手把手地带你搭建了 iOS 环境，以及 Appium 测试环境，并介绍了如何通过 Appium Inspector 来定位页面元素。

搭建好了测试环境后，我分别针对 Native App 和 Web App 这两类移动应用，基于 Appium 实现了两个测试用例，这也是我在这个专栏里面，为你实现的第一个移动应用的测试用例。虽然测试需求比较简单，但是你也可以从中体会到移动应用测试用例设计的思想、方法。

最后，本着知其然知其所以然的原则，我介绍了 Appium 的实现原理：它属于 C/S 架构，Appium Client 通过第三方库向 Appium Server 发起请求，Appium Server 接受请求，然后和移动平台上的代理工具打交道，代理工具在运行过程中不断接收来自 Appium Server 的请求，并解析出要执行的操作，最后调用移动平台原生的测试框架完成测试操作。

思考题

我在这篇文章里面举的例子都是基于 iOS 的，建议你基于 Android 分别实现一个 Web App 和 Native App 的测试用例。

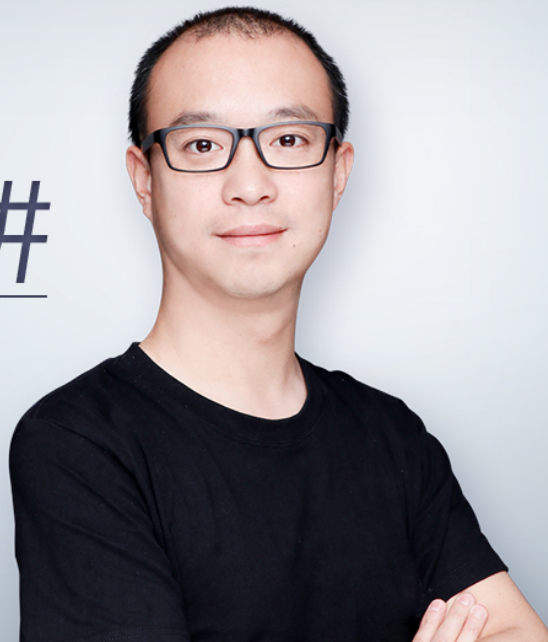
如果实现过程中，遇到了问题，或者有一些自己的想法，请给我留言讨论吧。

软件测试52讲

从小工到专家的实战心法

茹炳晟

eBay中国研发中心
测试基础架构技术主管



新版升级：点击「 请朋友读」，10位好友免费读，邀请订阅更有**现金**奖励。

© 版权归极客邦科技所有，未经许可不得传播售卖。页面已增加防盗追踪，如有侵权极客邦将依法追究其法律责任。

上一篇 20 | 与时俱进：浅谈移动应用测试方法与思路

下一篇 22 | 从0到1：API测试怎么做？常用API测试工具简介

精选留言 (12)

写留言



sylan215

2018-08-15

4

Appium 作为目前最主流的移动端自动化测试工具，应用的范围应该是相当广泛了，技术上也是相当成熟了，本次讲解的步骤非常详细，可以让初学者轻松上手，赞👍

欢迎沟通交流，公众号「sylan215」

展开 ▾



孙丢丢

2018-08-16

1

老师好~

Appium1.6.2 + iOS 10.1.1 inspector时定位不到xpath 提示的是：

(//XCUIElementTypeStaticText[@name= "AX error -25205"])[2]. google了一下感觉iOS 11系统可以解决这个问题 不过11系统还有其他的问题 wda的证书要每次都去手动信任一下 就导致跑不起来 陷入了死循环。不确定老师有没有碰到这个问题 期待老师的回...
展开 ∨



叶夏立

2018-08-15

👍 1

茹老师很好的给小白们普及了一下appium的基本知识，也让我好好回顾了一下appium👏



hi ! girl

2018-08-15

👍 1

老师，由于国内手机权限很多，执行appium时，安装的辅助apk不能自动安装完成，想问老师有什么好的想法解决？目前我的做法是：检测设备端是否存在appium相关app，存在，不安装；不存在，安装，所以首次安装都需要手动点击权限，第二次后执行不需要安装，但是部分手机运行时不会执行命令，此时会调起安装appium server的权限框，又该如何解决呢？

展开 ∨



emilymeng

2018-08-15

👍 1

老师讲的这节太及时了，刚好可以使用到。

展开 ∨



口水窝

2019-04-10

👍

今天看了原理，明天实践操作起来

展开 ∨

作者回复: 哈哈，希望可以成功运行



johnny

2019-01-10

👍

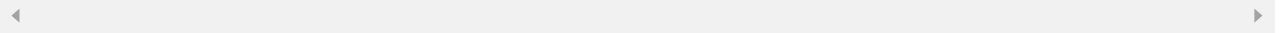
按照你说的步骤实现了。

第一次接触MAC OS、IDEA开发工具、Appium...

点赞。

展开 ▾

作者回复: 感谢支持, 看到第一个移动app的自动化测试跑起来的感觉一定很棒



小老鼠

2018-10-27



- 1、在VMWare MAC可以搭建这个环境吗?
- 2、MAC也应该支持Python



wfmedtest

2018-10-24



Appium搭建环境能否讲下

展开 ▾



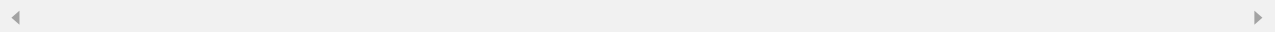
阿土白白...

2018-09-11



老师我想请教一个ios自动化的问题。appium 的inspector通过accessibility id 识别ios元素。有条用例, 我想检查下B页面某个元素的text 值是否是A页面传过来的, 但该元素的accessibility id就是它的text value。就好比, 一个元素是7, 去检查它是不是7, 好像没有意义呢。请问这种情况该如何处理呢?

作者回复: 可以考虑不同accessibility id作元素识别



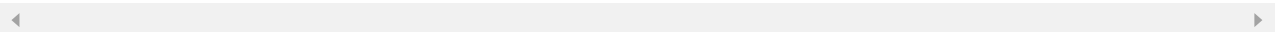
浮躁的社会

2018-08-15



谁有详细的appium使用文档吗? 网上找了一下都很零碎, 哪位大佬有给分享一下谢谢

作者回复: 可以参考官方文档, 不过是英文的, 但是个人觉得appium的官方文档写得不够友好, 尤其是对于初学者。





浮躁的社会

2018-08-15



网上找了一下并没有详细的appium的相关教程，都很零碎，请问有相关使用教程的文档吗